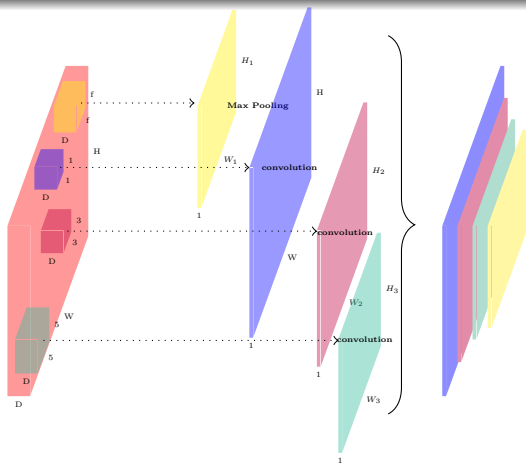
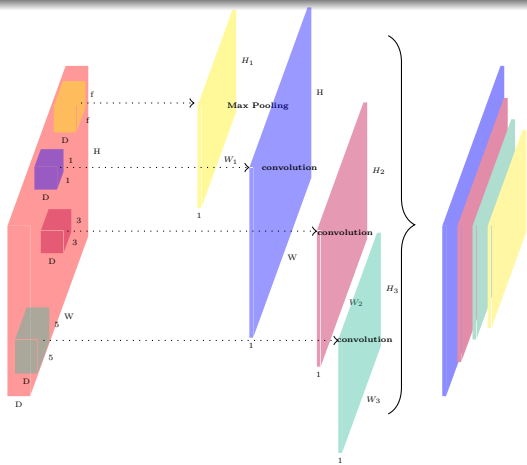


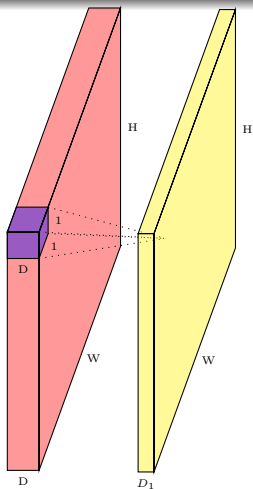
## Module: Image Classification continued (GoogLeNet and ResNet)



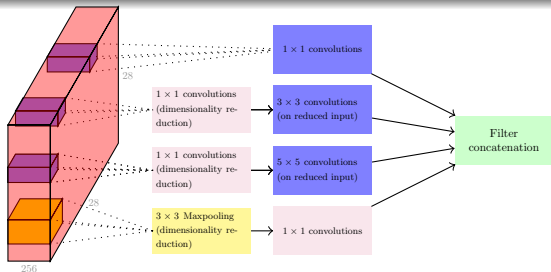
- Consider the output at a certain layer of a convolutional neural network
- After this layer we could apply a max-pooling layer
- Or a  $1 \times 1$  convolution
- Or a  $3 \times 3$  convolution
- Or a  $5 \times 5$  convolution
- **Question:** Why choose between these options (convolution, maxpooling, filter sizes)?
- **Idea:** Why not apply all of them at the same time and then concatenate the feature maps?



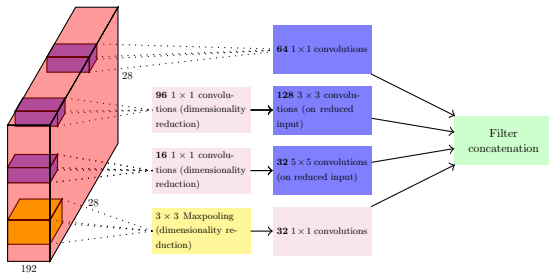
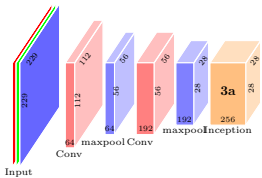
- Well this naive idea could result in a large number of computations
- If  $P = 0$  &  $S = 1$  then convolving a  $W \times H \times D$  input with a  $F \times F \times D$  filter results in a  $(W - F + 1)(H - F + 1)$  sized output
- Each element of the output requires  $O(F \times F \times D)$  computations
- Can we reduce the number of computations?

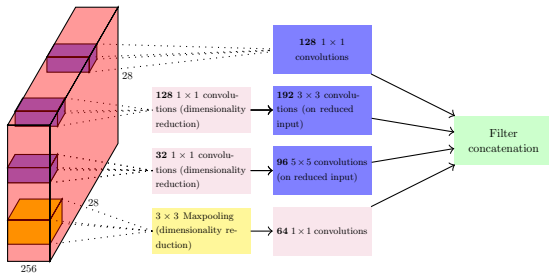
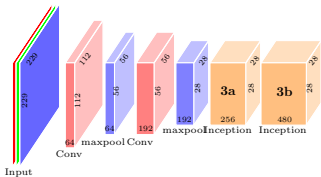


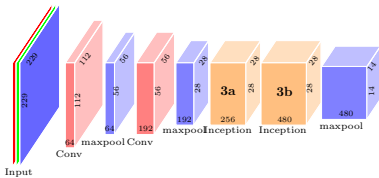
- Yes, by using  $1 \times 1$  convolutions
- Huh?? What does a  $1 \times 1$  convolution do ?
- It aggregates along the depth
- So convolving a  $D \times W \times H$  input with  $D_1$   $1 \times 1$  ( $D_1 < D$ ) filters will result in a  $D_1 \times W \times H$  output ( $S = 1, P = 0$ )
- If  $D_1 < D$  then this effectively reduces the dimension of the input and hence the computations
- Specifically instead of  $O(F \times F \times D)$  we will need  $O(F \times F \times D_1)$  computations
- We could then apply subsequent  $3 \times 3$ ,  $5 \times 5$  filter on this reduced output



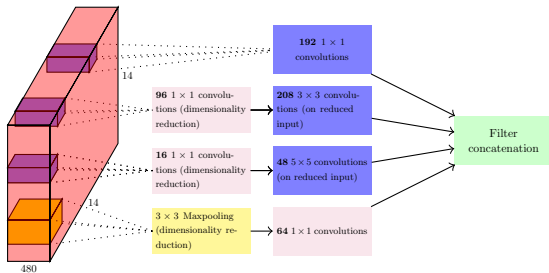
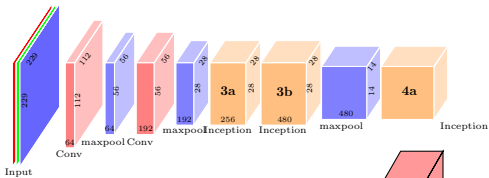
- But we might want to use different dimensionality reductions before the  $3 \times 3$  and  $5 \times 5$  filters
- So we can use  $D_1$  and  $D_2$   $1 \times 1$  filters before the  $3 \times 3$  and  $5 \times 5$  filters respectively
- We can then add the maxpooling layer followed by dimensionality reduction
- And a new set of  $1 \times 1$  convolutions
- And finally we concatenate all these layers
- This is called the **Inception module**
- We will now see **GoogLeNet** which contains many such inception modules

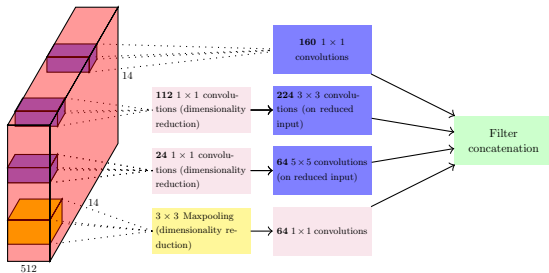
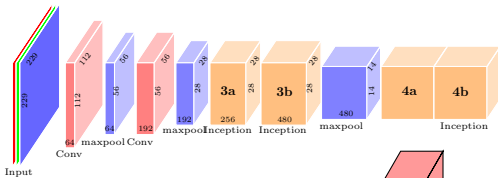


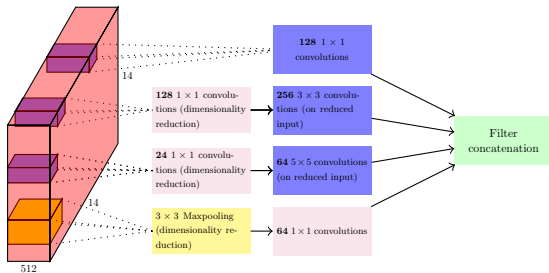
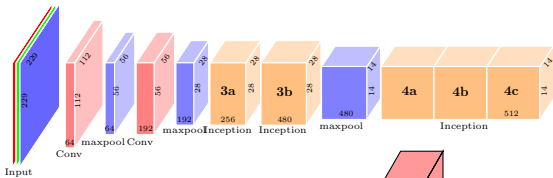


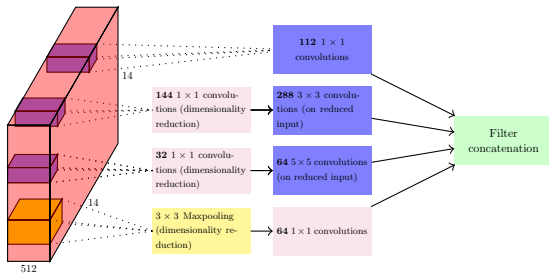
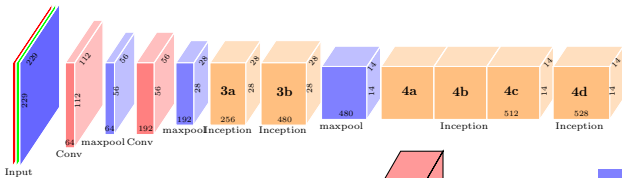


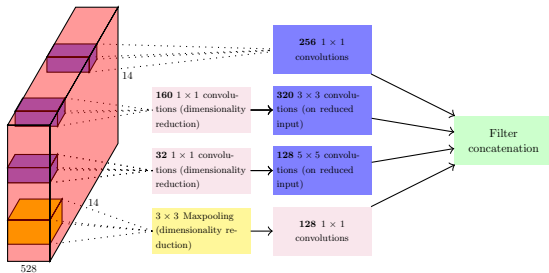
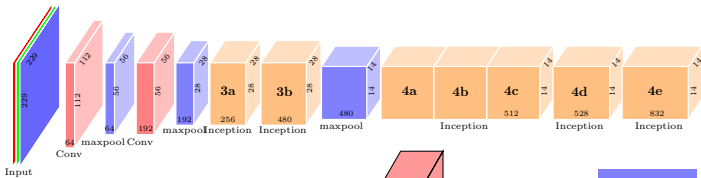


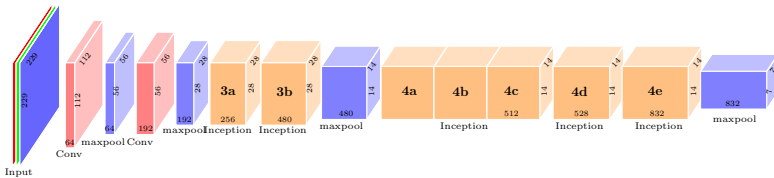


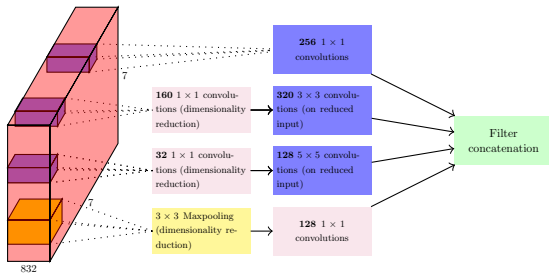
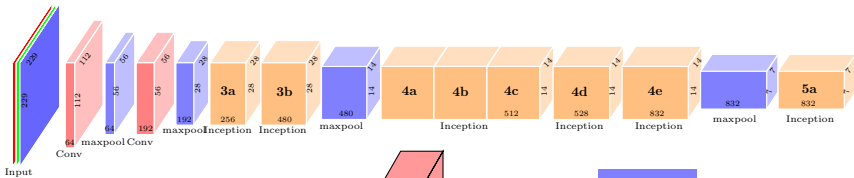


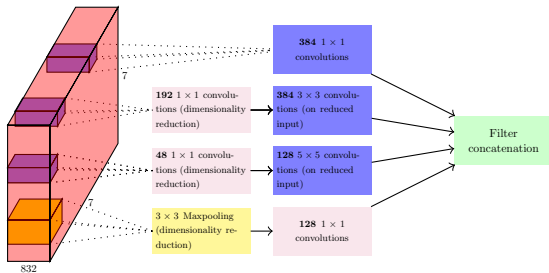
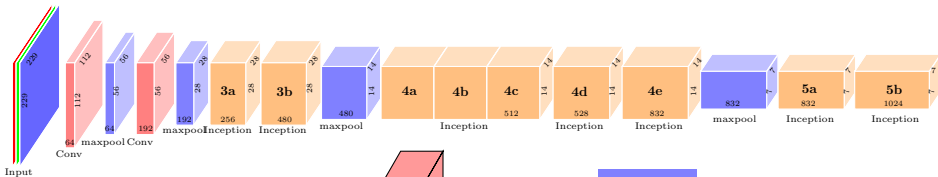




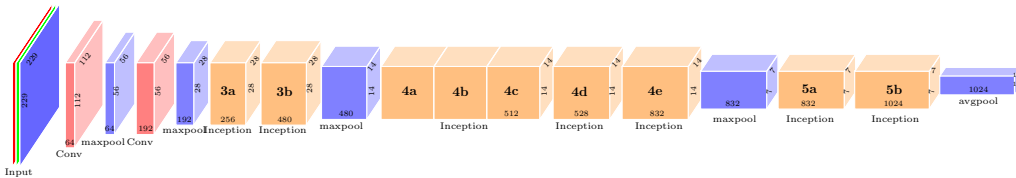


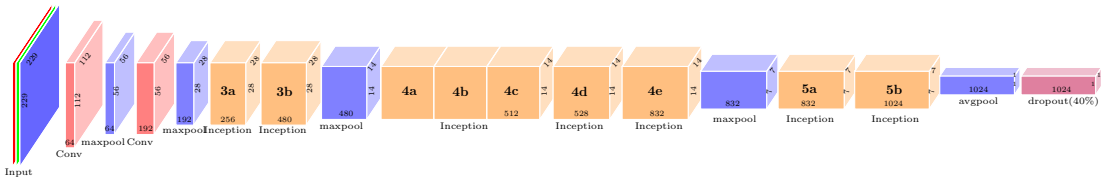


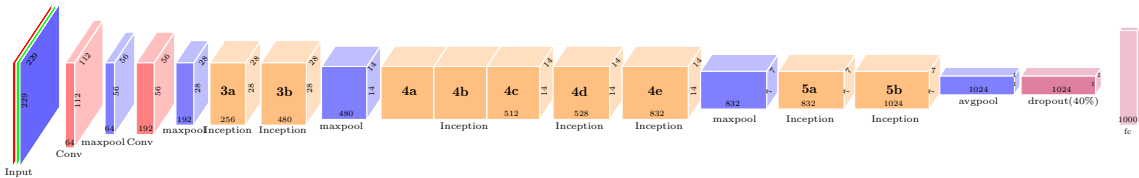


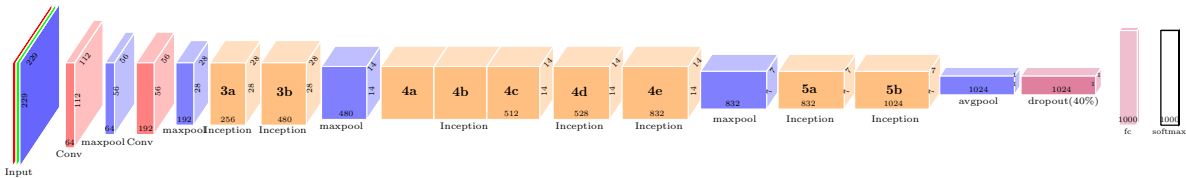


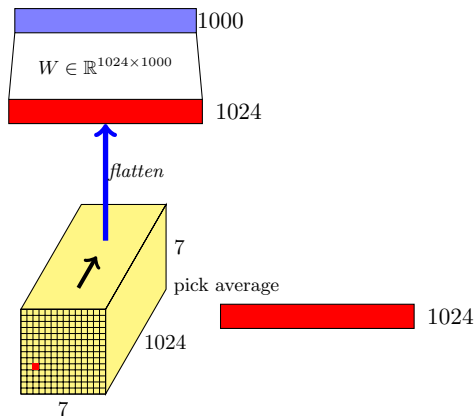




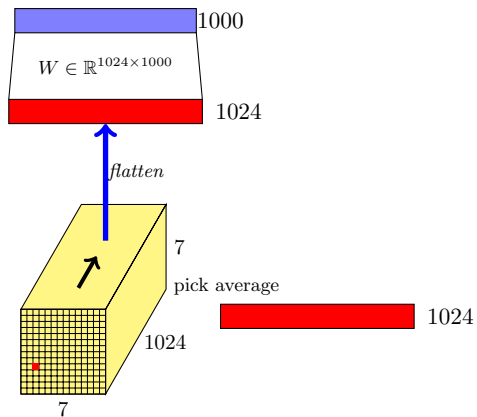






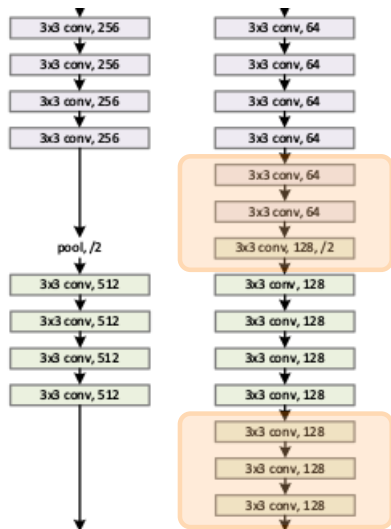


- **Important Trick:** Got rid of the fully connected layer
- Notice that output of the last layer is  $7 \times 7 \times 1024$  dimensional
- What if we were to add a fully connected layer with 1000 nodes (for 1000 classes) on top of this
- We would have  $7 \times 7 \times 1024 \times 1000 = 49M$  parameters
- Instead they use an average pooling of size  $7 \times 7$  on each of the 1024 feature maps
- This results in a 1024 dimensional output
- Significantly reduces the number of parameters



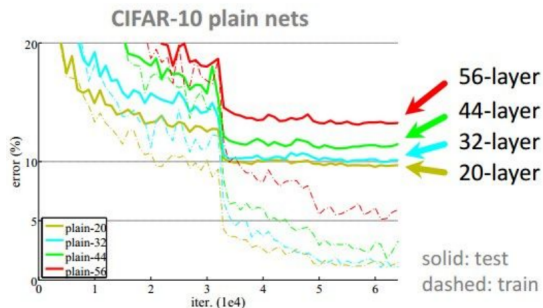
- $12\times$  less parameters than AlexNet
- $2\times$  more computations

- GoogLeNet
- ResNet

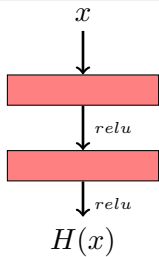


- Suppose we have been able to train a shallow neural network well
- Now suppose we construct a deeper network which has few more layers (in orange)
- Intuitively, if the shallow network works well then the deep network should also work well by simply learning to compute identity functions in the new layers
- Essentially, the solution space of a shallow neural network is a subset of the solution space of a deep neural network

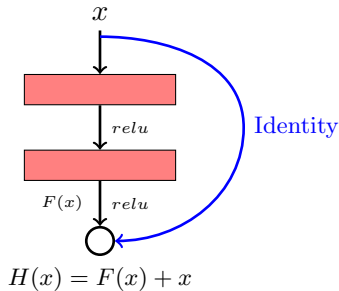


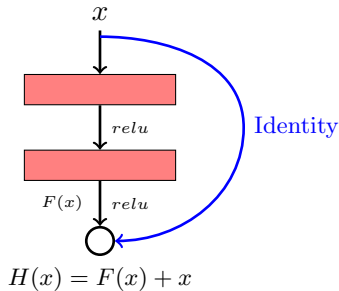
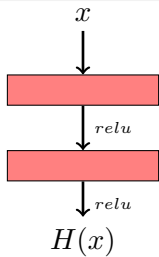


- But in practice it is observed that this doesn't happen
- Notice that the deep layers have a higher error rate on the test set



- Consider any two stacked layers in a CNN
- The two layers are essentially learning some function of the input
- What if we enable it to learn only a residual function of the input?





Residual Block

- Why would this help?
- Remember our argument that a deeper version of a shallow network would do just fine by learning identity transformations in the new layers
- This identity connection from the input allows a ResNet to retain a copy of the input
- Using this idea they were able to train really deep networks





ResNet, 152 layers

1<sup>st</sup> place in all five main tracks

- **ImageNet Classification:** “Ultra-deep” 152-layer nets
- **ImageNet Detection:** 16% better than the 2nd best system
- **ImageNet Localization:** 27% better than the 2nd best system
- **COCO Detection:** 11% better than the 2nd best system
- **COCO Segmentation:** 12% better than the 2nd best system



ResNet, 152 layers

### Bag of tricks

- Batch Normalization after every CONV layer
- Xavier/2 initialization from [He et al]
- SGD + Momentum(0.9)
- Learning rate:0.1, divided by 10 when validation error plateaus
- Mini-batch size 256
- Weight decay of  $1e-5$
- No dropout used